MEMORIA TÉCNICA

Dragones y Mazorcas

Buscador y comparador de juegos de mesa

Autora: Olatz González

Fecha: Abril 2026

Repositorio: github.com/olatzglez/Dragones_y_Mazorcas

Módulo: JavaScript — Proyecto Individual

Índice

1 Introducción y Contexto 3
1.1 Antecedentes y Definición del Problema 3
1.2 Estado del Arte y Justificación 3
2 Análisis de Requisitos 4
2.1 Requisitos Funcionales (RF) 4
2.2 Requisitos No Funcionales (RNF) 4
3 Planificación y Gestión del Tiempo 6
3.1 Desglose de Tareas y Estimaciones (Backlog) 6
3.2 Cronograma de Trabajo 6
4 Análisis Funcional y Flujos 7
4.1 Flujo de Usuario (User Flow) 7
4.2 Interfaz de Usuario (UI) 10
5 Arquitectura y Modelo de Datos 10
5.1 Modelo de Clases (POO) 10
5.2 Stack Tecnológico y Justificación 12
5.3 Estructura del Proyecto 13
6 Retos Técnicos y Soluciones 14
6.1 Consumo de la API de BoardGameGeek 14
6.2 Sistema de Favoritos con localStorage 14
6.3 Tabla Comparativa con Ordenación Dinámica 14
7 Conclusiones y Futuros Pasos 15

1 Introducción y Contexto

1.1 Antecedentes y Definición del Problema

El mercado de juegos de mesa ha experimentado un crecimiento notable en los últimos años, con catálogos que superan los 100.000 títulos registrados en plataformas como BoardGameGeek (BGG). Esta abundancia genera un problema real para el usuario: la dificultad de explorar, comparar y decidirse por un juego concreto sin una herramienta de apoyo adecuada.

Los jugadores se enfrentan habitualmente a las siguientes fricciones:

- Catálogo inmenso y difícil de filtrar sin una interfaz apropiada.
- Dispersión de información: precio, mecánicas, puntuaciones y reseñas se consultan en distintas plataformas.
- Falta de herramientas de comparación directa entre varios juegos candidatos.
- Ausencia de un espacio personal donde guardar y revisar los juegos de interés.

1.2 Estado del Arte y Justificación

Antes de abordar el desarrollo, se analizaron las alternativas disponibles:

- BoardGameGeek: La referencia en el sector, pero con una interfaz anticuada y poco intuitiva para usuarios no especializados.
- Tiendas online (Amazon, Philibert): Orientadas a la compra, sin herramientas de comparación ni gestión de listas personalizadas.
- Aplicaciones móviles (BG Stats, Boardgamegeek App): Enfocadas al registro de partidas, no al descubrimiento.

La propuesta: Dragones y Mazorcas nace para cubrir el hueco entre la información técnica de BGG y la experiencia de usuario moderna. Es una aplicación web 100% frontend que consume la API pública de BGG mediante un proxy local, presenta la información de forma clara y permite al usuario buscar, filtrar, guardar y comparar juegos en una sola interfaz.

2 Análisis de Requisitos

Se establecieron los siguientes requisitos formales para garantizar el cumplimiento de los objetivos del proyecto.

2.1 Requisitos Funcionales (RF)

ID	Nombre	Descripción
RF-01	Búsqueda por nombre	El sistema permite buscar juegos por nombre mediante la API de BGG. Los resultados se muestran dinámicamente sin recargar la página.
RF-02	Búsqueda por ID	El usuario puede buscar un juego concreto introduciendo su ID de BoardGameGeek para obtener sus datos detallados directamente.
RF-03	Filtrado avanzado	Una vez realizada una búsqueda, se activan filtros por número de jugadores, edad mínima, duración máxima, categoría, complejidad y rango de años.
RF-04	Gestión de favoritos	El usuario puede añadir y eliminar juegos de su lista de favoritos, que se persiste en localStorage entre sesiones.
RF-05	Tabla comparativa	La página de favoritos incluye una tabla comparativa con los datos clave de todos los juegos guardados, con ordenación por columna.
RF-06	Exportación de datos	El usuario puede exportar la tabla comparativa de favoritos en formato PDF o CSV.
RF-07	Landing page	La página de inicio (index.html) presenta el proyecto, sus funcionalidades y permite acceder a las distintas secciones de la app.

2.2 Requisitos No Funcionales (RNF)

ID	Nombre	Descripción
RNF-01	Diseño responsive	La interfaz debe adaptarse correctamente a dispositivos móviles, tablets y escritorio. Se han definido breakpoints en 768px y 480px.
RNF-02	Accesibilidad mínima	Los botones e interactivos tienen un tamaño mínimo de 44x44px para cumplir las recomendaciones de accesibilidad táctil (WCAG).
RNF-03	Persistencia de datos	Los favoritos se almacenan en localStorage y permanecen disponibles al cerrar y reabrir el navegador.
RNF-04	Arquitectura modular	El código JavaScript se organiza en módulos ES6 (import/export) separando clases, servicios de API, y componentes de DOM.

ID	Nombre	Descripción
RNF-05	Modo oscuro	La app incluye un sistema de tema claro/oscuro basado en variables CSS, con persistencia de la preferencia en localStorage.
RNF-06	Usabilidad	El sistema de diseño unificado (variables CSS, sistema de botones btn/btn-primary/btn-secondary) garantiza coherencia visual en todas las páginas.

3 Planificación y Gestión del Tiempo

3.1 Desglose de Tareas y Estimaciones (Backlog)

Se estimó una capacidad de trabajo de 6 horas efectivas diarias. Se aplicó un factor de seguridad del 20% sobre las estimaciones para cubrir imprevistos técnicos.

Fase	Ticket	Descripción Técnica	Estimación	Dep.
Diseño	Definición y arquitectura	Wireframes, estructura de clases, flujo de usuario y diseño de tokens CSS.	8h	—
Backend	Setup servidor proxy	Configuración de Node.js + Express como proxy para la API de BGG (evitar CORS).	4h	Diseño
POO	Clases del dominio	Implementación de BoardGame, Favorites y ApiService con herencia y encapsulación.	10h	Setup
Frontend	Páginas y DOM	Desarrollo de index.html, test-api.html y favorites.html con manipulación dinámica del DOM.	14h	POO
Frontend	Sistema CSS	Creación del sistema de diseño: variables, botones, componentes, dark mode, responsive.	10h	Diseño
Feature	Favoritos + Tabla	Sistema de favoritos con localStorage, tabla comparativa con ordenación y exportación PDF/CSV.	8h	DOM
Cierre	QA y limpieza	Testing manual de flujos críticos, corrección de bugs, revisión de código y commit final.	6h	Todo
Docs	Memoria + README	Redacción de la memoria técnica y actualización del README del repositorio.	5h	—

3.2 Cronograma de Trabajo

El desarrollo se organizó en dos semanas con las siguientes fases principales:

- Semana 1 (Días 1-3): Diseño de arquitectura, setup del servidor proxy y definición del modelo de clases.
- Semana 1 (Días 4-5): Implementación de las clases POO (BoardGame, Favorites, Api Service) y primeras llamadas a la API.
- Semana 2 (Días 6-8): Desarrollo del frontend, páginas HTML, manipulación del DOM y sistema de favoritos.
- Semana 2 (Días 9-10): Sistema CSS, dark mode, tabla comparativa, exportación y QA final.

4 Análisis Funcional y Flujos

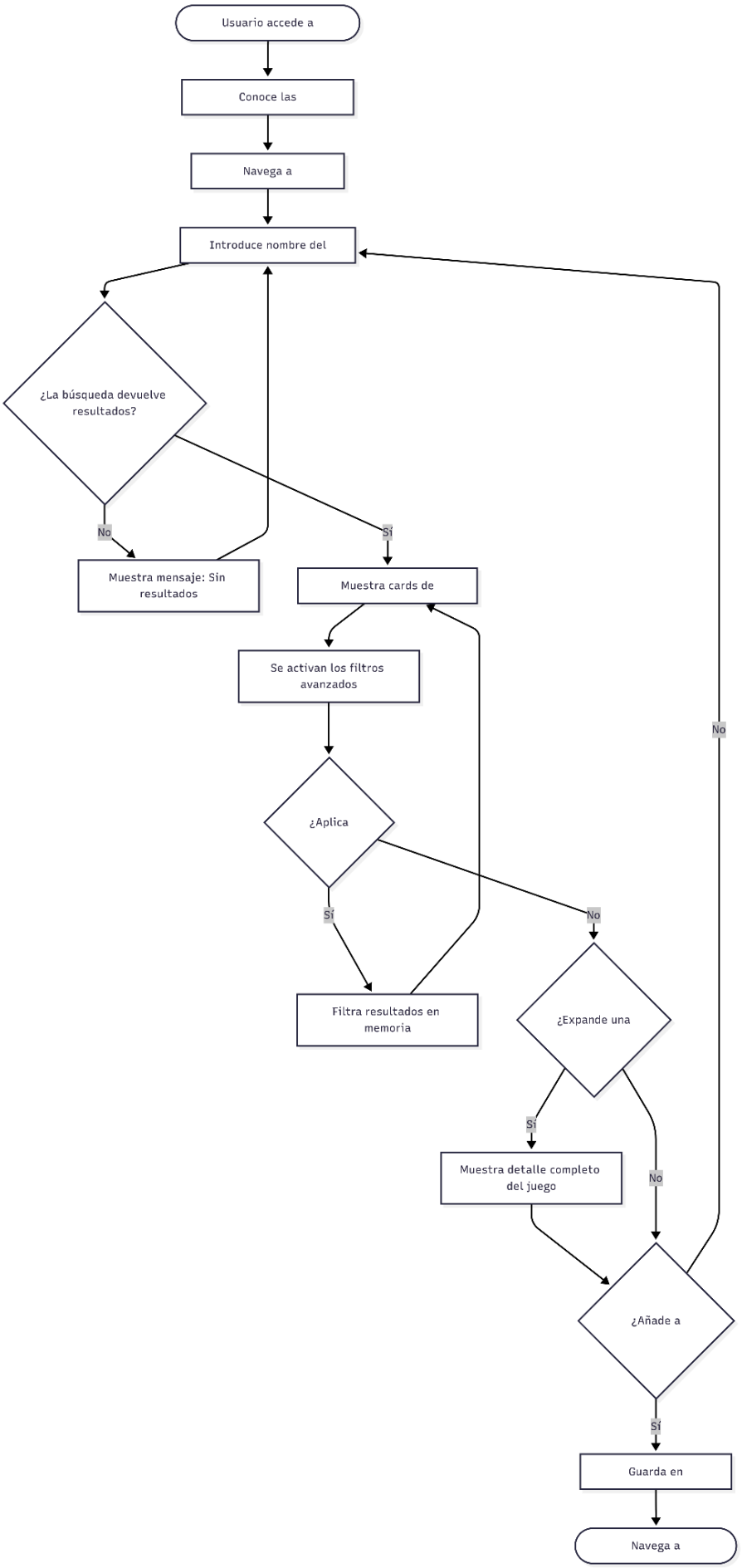
4.1 Flujo de Usuario (User Flow)

La aplicación presenta dos flujos principales según el punto de entrada del usuario:

Flujo de búsqueda y descubrimiento:

1. El usuario accede a la landing page (index.html) y conoce las funcionalidades.
2. Navega a la página de exploración (test-api.html).
3. Introduce un nombre de juego en la barra de búsqueda y ejecuta la búsqueda.
4. Se activan los filtros avanzados (jugadores, edad, duración, categoría, complejidad, año).
5. El usuario puede expandir cualquier card para ver el detalle completo del juego.
6. Puede añadir el juego a su lista de favoritos pulsando el botón correspondiente.

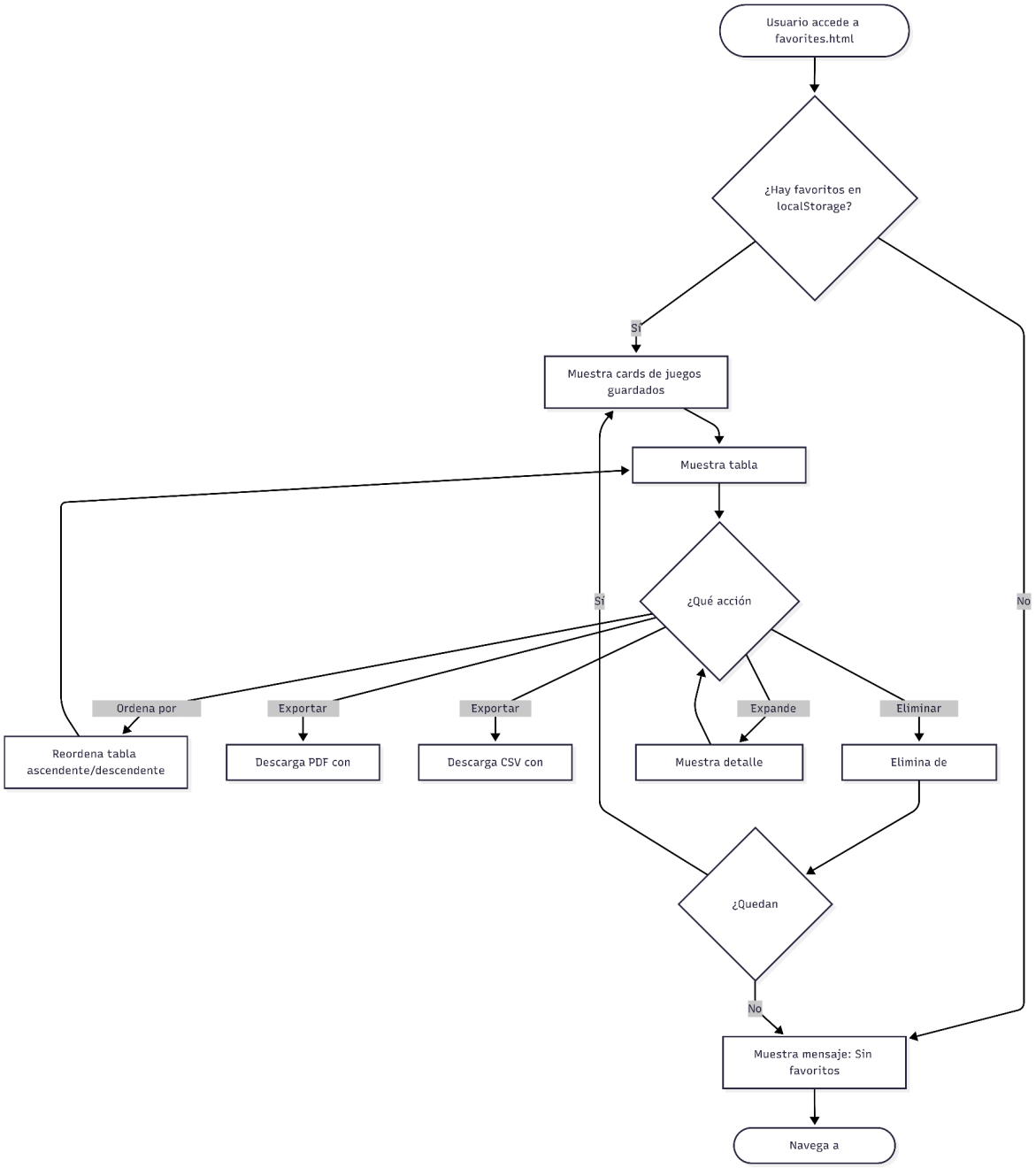
Enlace al Flujo de Bśuqueda y descubrimiento:
https://mermaid.ai/d/263bef87-25ae-411a-80e9-6b3be6e1cd17


Usuario accede a
Conoce las
Navega a
Introduce nombre del
¿La búsqueda devuelve resultados?
No
Sí
Muestra mensaje: Sin resultados
Muestra cards de
Se activan los filtros avanzados
¿Aplica
Sí
No
Filtra resultados en memoria
¿Expande una
Sí
No
Muestra detalle completo del juego
¿Añade a
No
Sí
Guarda en
Navega a

Flujo de gestión de favoritos:

7. El usuario accede a la página de favoritos (favorites.html).
8. Ve todos sus juegos guardados organizados en cards expandibles.
9. Puede consultar la tabla comparativa y ordenar los juegos por cualquier columna.
10. Puede exportar la lista en PDF o CSV.
11. Puede eliminar juegos de la lista, con actualización inmediata de la tabla.

Enlace a Flujo de Gestión de Favoritos:
https://mermaid.ai/d/a6e63af5-2fa6-4541-9e48-e98842559512



4.2 Interfaz de Usuario (UI)

Se han desarrollado tres páginas principales con diseño coherente basado en el sistema CSS compartido:

- index.html (Landing page): Hero con imagen de fondo, sección de pasos, categorías, funcionalidades y CTA final. Header y footer dinámicos compartidos con el resto de páginas.
- test-api.html (Explorador): Barra de búsqueda, panel de filtros avanzados, listado de resultados en cards expandibles con detalle completo de cada juego.
- favorites.html (Mis favoritos): Listado de juegos guardados en cards colapsables, y tabla comparativa con ordenación por columna, exportación PDF y CSV.

Decisiones de diseño destacadas:

- Sistema de botones unificado: clases btn, btn-primary, btn-secondary, btn-tertiary y btn-danger, sin tamaños por clase sino por contexto CSS.
- Secciones plegables en favoritos con animación CSS (max-height + opacity) para mejorar la usabilidad en listas largas.
- Modo oscuro implementado mediante sobreescritura de variables CSS bajo body.dark-mode, con persistencia en localStorage.
- Header sticky con max-width de 1200px para garantizar legibilidad en pantallas grandes.

5 Arquitectura y Modelo de Datos

5.1 Modelo de Clases (POO)

La aplicación aplica Programación Orientada a Objetos mediante tres clases principales que encapsulan la lógica del dominio:

BoardGame

Clase central que representa un juego de mesa. Recibe los datos crudos de la API y los transforma en una instancia con propiedades tipadas y métodos de presentación.

- Propiedades: id, name, year, image, categories[], mechanics[], publishers[], language[].
- Métodos: getPlayers(), getPlaytime(), getMinAge(), getRating(), getVotes(), getWeight(), getRank(), getDescription().
- Cada getter encapsula la lógica de extracción y formateo del dato desde la respuesta XML de BGG.

Favorites

Clase que gestiona la colección de juegos favoritos del usuario. Actúa como capa de abstracción sobre localStorage.

- Propiedades: items[] (instancias BoardGame).
- Métodos: add(game), remove(id), getAll(), isFavorite(id), count().
- La persistencia se realiza serializando los objetos BoardGame a JSON en localStorage y deserializándolos al instanciar la clase.

ApiService

Clase de servicio que centraliza todas las llamadas a la API. Abstrae la comunicación con el servidor proxy local que actúa como intermediario con la API XML de BoardGameGeek.

- Métodos: searchGamesSimplified(query), getGameSimplified(id).
- Utiliza fetch con async/await y maneja los errores de red mediante try/catch.

Relación entre clases: ApiService obtiene los datos brutos, BoardGame los modela y Favorites los almacena. Las páginas HTML consumen estas tres clases a través de módulos ES6 (import/export).

5.2 Stack Tecnológico y Justificación

Tecnología	Uso	Justificación
HTML5	Estructura de páginas	Estándar web. Semántica correcta con elementos como <header>, <main>, <section>, <footer>.
CSS3 + Variables	Sistema de diseño	Variables CSS para theming y modo oscuro. Arquitectura modular en ficheros independientes (header, footer, components, landing, darkmode).
JavaScript ES6	Lógica de la app	Módulos nativos (import/export), clases, async/await, destructuring y operadores modernos sin frameworks adicionales.
Fetch API	Consumo de API	API nativa del navegador para peticiones HTTP asíncronas. Se usa con async/await para mayor legibilidad.
localStorage	Persistencia	Almacenamiento de favoritos y preferencia de tema (dark/light) entre sesiones sin necesidad de backend.
Node.js + Express	Servidor proxy	Necesario para evitar los errores CORS al consumir la API XML de BoardGameGeek desde el navegador.
BoardGameGeek API	Fuente de datos	API pública y gratuita con datos de más de 100.000 juegos de mesa: puntuaciones, mecánicas, editoriales, idiomas y más.
jsPDF + AutoTable	Exportación PDF	Librería cargada via CDN para generar el PDF de la tabla comparativa directamente en el navegador.

5.3 Estructura del Proyecto

El repositorio sigue una organización modular que separa claramente estilos, lógica de negocio y vistas:

```
IndividualJS/
├── docs/
│   ├── memoria_dragones_mazorcas.pdf          # Memoria técnica del proyecto
│   └── README.md
├── js/
│   ├── api/
│   │   └── apiService.js      # Servicio de consumo de API
│   ├── classes/
│   │   ├── Game.js            # Clase base
│   │   ├── BoardGame.js       # Clase del dominio (hereda de Game)
│   │   └── Favorites.js       # Gestión de favoritos con localStorage
│   └── dom/
│       ├── header.js          # Componente header dinámico
│       ├── footer.js          # Componente footer dinámico
│       └── filters.js         # Lógica de filtros avanzados
├── src/
│   ├── assets/images/         # Logos y recursos gráficos
│   └── css/
│       ├── styles.css         # Variables CSS globales
│       ├── components.css     # Botones y componentes UI
│       ├── header.css         # Estilos del header
│       ├── footer.css         # Estilos del footer
│       ├── landing.css        # Estilos de la landing page
│       └── darkmode.css       # Sobreescritura de variables para modo oscuro
├── index.html                 # Landing page
├── buscador.html              # Explorador de juegos
├── favorites.html             # Página de favoritos
├── server.js                  # Servidor proxy Node.js + Express
├── .env                       # Token de la API de BGG (no incluido en el repo)
├── .gitignore
└── README.md
```

6 Retos Técnicos y Soluciones

6.1 Consumo de la API de BoardGameGeek

Situación: La API de BGG es una API XML que no permite peticiones directas desde el navegador debido a las restricciones de CORS. Además, devuelve datos en formato XML que el navegador no procesa nativamente como JSON.

Investigación: Se intentó inicialmente consumir la API directamente desde el frontend, obteniendo errores de CORS en todos los entornos de producción.

Solución implementada: Se creó un servidor proxy con Node.js y Express que actúa de intermediario. El servidor recibe la petición del frontend, la reenvía a la API de BGG, parsea la respuesta XML y la devuelve al cliente como JSON limpio. Esto centraliza también la lógica de transformación de datos en ApiService.

6.2 Sistema de Favoritos con localStorage

Situación: Al guardar instancias de la clase BoardGame en localStorage, la serialización a JSON pierde los métodos del prototipo. Al recuperar los datos, los objetos eran POJOs sin acceso a getPlayers(), getRating(), etc.

Solución implementada: La clase Favorites, al recuperar los datos de localStorage mediante getAll(), deserializa cada objeto JSON y crea una nueva instancia de BoardGame con esos datos. De esta forma, los métodos del prototipo están siempre disponibles sin necesidad de llamar de nuevo a la API.

```
getAll() {  const raw = JSON.parse(localStorage.getItem(this.key) || '[]');  return raw.map(data => new
BoardGame(data)); // Rehidrata las instancias}
```

6.3 Tabla Comparativa con Ordenación Dinámica

Situación: Los getters de la clase BoardGame devuelven strings con formato legible (ej: '2–4 jugadores', '+12 años', '30–150 min'), pero para ordenar la tabla se necesitan valores numéricos.

Solución implementada: Se creó el helper parseFirst() que extrae el primer número de cualquier string mediante una expresión regular. Esto permite ordenar correctamente por cualquier columna sin modificar la lógica de presentación de BoardGame, y garantiza que los valores sin dato (—) siempre aparezcan al final de la ordenación.

```
function parseFirst(str) {  if (str == null) return Infinity; // Los vacíos van al final  const match =
String(str).match(/[\d,.]+/);  return match ? parseFloat(match[0].replace(',', '.')) : Infinity;}
```

7 Conclusiones y Futuros Pasos

El desarrollo de Dragones y Mazorcas ha permitido consolidar y aplicar de forma integrada los conceptos fundamentales del módulo de JavaScript: Programación Orientada a Objetos, manipulación del DOM, consumo asíncrono de APIs y uso de localStorage.

Aprendizajes técnicos destacados:

- La importancia de la rehidratación de objetos al recuperar instancias de clases desde localStorage, ya que JSON.parse pierde el prototipo.
- El uso de un servidor proxy Node.js como solución elegante al problema de CORS con APIs externas que no admiten peticiones desde el navegador.
- La ventaja de un sistema CSS basado en variables (:root) para implementar el modo oscuro sin duplicar estilos, simplemente sobreescribiendo las variables bajo body.dark-mode.
- La separación de responsabilidades en módulos ES6 (clases, servicios, componentes DOM) facilita el mantenimiento y la escalabilidad del código.

Requisitos opcionales implementados:

- Modo oscuro/claro con persistencia en localStorage.
- Exportación de datos a PDF y CSV desde la tabla comparativa.

Mejoras planificadas para futuras versiones:

12. Implementar paginación en los resultados de búsqueda para catálogos extensos.
13. Añadir la posibilidad de añadir notas o puntuaciones personales a cada favorito.
14. Crear una vista de detalle individual por juego con URL propia (routing).
15. Integrar autenticación simulada para gestionar múltiples perfiles de usuario.
16. Desplegar la aplicación completa en un servicio de hosting (Vercel, Netlify o Railway).